

ComicCraft - AI Comic Story Creator using Gemini Models

5. System Design

Technology Track: Generative AI with Google

1. Architecture Overview

ComicCraft uses a three-layer architecture: a React frontend for input and preview, a Node.js/Express backend that orchestrates all Gemini API calls, and a comic assembly step that merges generated panel images and captions into one finished strip.

2. System Flow

- The frontend collects the story idea, art style, and panel count from the user.
- The backend validates the request and passes the story to the Scene Planner.
- The Scene Planner (Gemini text model) breaks the story into an ordered set of panel scenes with captions.
- The Art Generator (Gemini image model) creates artwork for each scene, guided by a fixed style description.
- The Strip Builder assembles all panel images and captions into one comic layout.
- The finished comic is returned to the frontend for preview, panel regeneration, and download.

3. Module Breakdown

Module	Responsibility
Prompt Intake	Collects and validates the story text, style, and panel count.
Scene Planner	Uses Gemini text generation to create a panel-by-panel script.
Caption Generator	Writes a short dialogue/caption line for each panel.
Art Generator	Calls Gemini image generation per panel using a locked style/character description.
Strip Builder	Arranges panel images and captions into a single comic strip.
Export Service	Converts the final layout into a downloadable file.

4. Data Flow

Story text enters the Scene Planner, which returns scene descriptions and captions. Each scene description is sent to the Art Generator along with a fixed style/character phrase, producing one image per panel. The Strip Builder then merges these images and captions into one strip, which is returned to the frontend as the final result.

5. UI/Screen Design

- Start Screen — story input box, style dropdown, panel-count selector, Generate button.
- Progress Screen — status updates while scenes and artwork are generated.
- Result Screen — final comic preview, per-panel Regenerate option, Download button.
- History Screen (optional) — list of previously generated comics.

6. Data Model (for optional history)

Field	Type	Description
userId	String	Identifies the comic's owner
storyPrompt	String	Original story idea submitted
style	String	Art style selected for the comic
panelImageUrls	Array<String>	Links to each generated panel's artwork
generatedAt	Timestamp	Date and time of generation

7. Technology Stack

- Frontend: React.js, HTML5, CSS3
- Backend: Node.js, Express.js
- AI: Google Gemini API — text and image generation
- Storage: Firebase Firestore (optional)
- Hosting: Firebase Hosting or Google Cloud Run

8. Design Notes

- A single, detailed style/character description is reused for every panel's image prompt in a session, keeping the art visually consistent.
- Regenerating a panel re-runs only the Art Generator for that scene, not the entire pipeline.
- The Strip Builder's layout logic automatically adjusts to different panel counts.

9. Summary

This design cleanly separates story planning, artwork generation, and final assembly into independent, testable modules, keeping the system easy to build, debug, and extend.